

One Account, Many Machines

Designing the Multi-Host SARSI Console

Per-Host Agents, Host-Indexed Evidence, Channels That Can Drop,
and What a Machine Takes With It When It Leaves

Chengshuai Yang

NextGen PlatformAI C Corp.

`spiritai@platformai.org`

August 1, 2026

Abstract

The companion design *One Chat Box, Five Agents* specifies a manager-fronted console with five agents, four inter-agent channels, and per-agent evidence-linked self-models. It contains an assumption it never states: that the fleet lives on one host. Every channel is a function call, every report arrives, every measurement is comparable with every other, and the machine agent is a singular noun.

That assumption has failed in deployment. An account now owns several machines; a machine joins an account and can leave it; the host that serves the registry is not a member of the fleet it serves; and the account's machine agent has already migrated from one host to another while its sessions were copied across by hand. This paper is the design of the console under those conditions.

Three things that were free become load-bearing. **Evidence is host-indexed:** a coordinate measured on one machine is not evidence about another, and the natural account-level summary — a mean over machines — is not monotone in fleet health, so an account-level self-state must be a vector over machines rather than a scalar over the account. **Channels can drop:** a channel that may fail to deliver is a different object from one that cannot, and the companion's leak test — project twice, assert containment — is satisfied vacuously by a dead link, so containment needs a liveness companion or silence reads as health. **Membership is revocable:** what a machine takes with it when it leaves is a design decision, and the capability posterior is the part that must not travel, because it was earned under a tool environment the receiving host does not necessarily satisfy.

The roster re-partitions along a scope axis rather than a role axis: two agents are per account, one is per host and therefore plural, one is per task, and exactly one legitimately spans hosts — which makes it the only place cross-machine coordination is designed rather than forbidden. The chat box does not change. The user still meets one of them, and the manager still routes on organizational awareness rather than capability — a position the companion argued from parsimony and this paper obtains as a consequence, since a manager separated from a host by a network cannot hold that host's state and must route on ownership or ask.

Keywords: SARSI; multi-agent systems; distributed systems; functional self-awareness; fleet management; agent governance; partial failure; system design.

Contents

1	Introduction	4
1.1	What the user meets, unchanged	4

1.2	The assumption the companion did not state	4
1.3	Three things that become load-bearing	4
1.4	Results	4
1.5	Scope	5
2	The Account and the Machine	5
2.1	Definitions	5
2.2	A label is not an identifier	6
2.3	Serving the registry is not membership	6
2.4	Identity gains a host index	7
3	Evidence Is Host-Indexed	7
3.1	A measurement is about the machine that took it	7
3.2	Why the fleet mean is not a health measure	7
3.3	Capability is bound to a tool environment	8
3.4	Cold start is per machine	9
4	Channels That Can Drop	9
4.1	The four channels, re-typed	9
4.2	Containment is satisfied by a dead link	10
4.3	Attributable silence	11
4.4	A control you cannot deliver must fail visibly	11
5	Isolation You Do Not Configure	12
6	Scope, Not Role, Is the Partition	13
6.1	The learning agent must be per account	13
6.2	The machine agent is plural, and each is a peer of the others	13
7	The Machine SARSI Agent, Respecified	14
7.1	Self-state	14
7.2	What it reports, and how often	14
7.3	Failure modes multi-host adds	15
8	The Working-Process Agent: The Designed Exception	15
8.1	Why this is not a peer edge	15
8.2	What the process agent must model that no one else does	16
9	The Chat Box, Unchanged and Harder	17
9.1	“Which machine?” is an organizational question	17
9.2	Routing without host evidence	17
10	Owner Controls Across a Fleet	17
10.1	Why there is no machine-level governor	18
10.2	The new obligation	18
11	Failure Modes This Version Adds	18
12	Build Order	19

13 Conclusion	19
A Invariant Checklist: Multi-Host Additions	20
B Deployment Status Summary	20

PART I — WHAT CHANGES WHEN THE FLEET SPANS HOSTS

1 Introduction

1.1 What the user meets, unchanged

A user arriving at the console still meets one chat box. Nothing in this paper changes that, and the reader should hold it fixed while everything behind it becomes plural. The surface is the companion design’s first result and it survives intact: what the user reads is a rendering of the manager’s own bounded workspace, and what the user writes is evidence entering the manager’s admission rule at owner-declaration authority [1].

What changes is what stands behind the box. In the companion there are five agents on one machine. Here there is an *account*, and an account owns machines — an indeterminate number of them, each with its own operating system, its own installed tools, its own context, and its own capacity to be unreachable.

1.2 The assumption the companion did not state

The unstated premise

Every specification in *One Chat Box, Five Agents* is written as though the five agents share a host. The machine agent is “the” machine agent. A typed report “carries” state upward. A projection $\text{Proj}_{\text{mac}}(W^M)$ is evaluated as though both workspaces are readable at the same instant. None of these is wrong on one host, and none survives two.

The premise is not a lapse; it was true when the companion was written. It stopped being true in the ordinary way that deployment assumptions stop being true: the owner acquired a second machine, moved the account’s machine agent onto it, and discovered that the console had opinions about which host was “the” host that nobody had ever written down.

Three deployment facts fix the new setting, and each one breaks something the companion assumed:

1. **An account has a machine list, and a machine can leave.** A machine joins an account at install time; `unlink` removes it. The account, in both directions, is taken from the caller’s key and never from the request body — a machine may state which machine it is, never which account it belongs to.
2. **Serving the registry is not membership in the fleet.** The production host serves the account registry for every machine while not being a member of the fleet it serves. The two were conflated for as long as there was one host, because on one host they are the same host.
3. **Agents outlive hosts.** The account’s machine agent moved from one host to another. Its *sessions* were carried across as files. Its *evidence* — what it had verified it could do — was not, and this paper argues that was correct rather than merely expedient.

1.3 Three things that become load-bearing

1.4 Results

Four results

1. **An account-level self-state is a vector over machines, not a scalar over the account.** No scalar aggregate of a host-indexed integrity coordinate is monotone in fleet health: adding an idle machine raises a fleet mean while changing nothing about the machine doing the work (Proposition 1). This is the companion’s own “vector, not level” argument, one level up, and it was not free — the natural

Table 1: What one host made free, and what it costs once there are two.

Property	Free on one host	Load-bearing on many
Comparability of evidence	Every measurement comes from the same machine, so coordinates are commensurable	A reading is indexed by the host that took it; aggregation across hosts is a modelling decision with a wrong default (§3)
Delivery	A channel is a function call; sending is receiving	A channel may drop, and absence of a report is ambiguous between <i>nothing happened</i> and <i>nothing arrived</i> (§4)
Membership	The fleet is whatever is running	Membership is a revocable relation, so identity, authority and history all need a scope (§2)

implementation is an average.

2. Containment is not a leak test across a network. The companion’s monotonicity check — project twice, assert containment — holds vacuously when nothing arrives, so a partition passes it (Proposition 3). Multi-host requires a liveness companion: every absence must be attributable to *not reported* or *not delivered*, or silence is read as health.

3. Capability does not migrate; history may. A capability posterior is evidence gathered under a tool environment. Transferring it to another host imports outcomes obtained under conditions the receiving host does not necessarily satisfy (Proposition 2). Episodic and procedural memory may cross with provenance; the posterior is re-earned.

4. Exactly one agent legitimately spans hosts. Re-partitioning the roster by scope rather than by role leaves the working-process agent as the sole agent whose object is a workflow that crosses machines. It is therefore the only place cross-machine coordination is designed rather than prohibited — and the design must show why this does not reinstate a peer edge (§8).

1.5 Scope

This document designs the console. It does not restate the self-awareness architecture [3, 5], the hierarchical fleet semantics [2, 4], or the per-agent specification the companion gives in full [1]. Where this paper contradicts the companion, it says so and gives the reason; where it merely adds a host index, it says that too, because most of the companion survives unmodified and a design paper that implied otherwise would be overstating its own contribution.

Deployment status is reported per section against the running console, in the manner of the deployment edition [6]: what is built, what is specified, and what is neither.

2 The Account and the Machine

2.1 Definitions

Definition 1 (Account). *The unit of ownership. An account is a person or organisation holding credentials, grants, and a machine list. Authority originates here, and only here.*

Definition 2 (Machine). *A host running an installation of the console. A machine has a stable identifier, a mutable human-readable label, and at most one account membership at a time.*

Definition 3 (Membership). *The relation between an account and a machine. It is created at install or by an explicit link, revoked by an explicit unlink, and it is the scope within which every host-indexed coordinate in this paper is meaningful.*

Rule 1 (The account comes from the key, never from the body). *On every membership operation — link, unlink, report — the account is derived from the calling credential and never read from the request. A machine may assert which machine it is. It may not assert which account it belongs to.*

The rule looks like an implementation detail and is not. Membership is the scope of authority: if a machine could name its own account, a compromised or merely misconfigured host could enroll itself into a fleet whose grants it would then inherit. The asymmetry — self-identification permitted, self-enrollment forbidden — is the same shape as the companion’s ban on self-certification, applied to the join operation rather than to a verdict.

2.2 A label is not an identifier

A machine carries both an identifier and a label, and the design must keep them distinct. The identifier is stable and machine-generated; the label is set by the owner and exists so that a person can tell two machines apart in a list. A label is therefore *owner declaration* on the evidence ladder — authoritative for what the owner wants the machine called and for nothing else.

The failure that motivates the distinction

Unlinking was first implemented as re-registering with an empty label. It did not remove the machine; it renamed the row to the default. From the account’s view the fleet still contained the host, now anonymous. A deletion expressed as a write to a display field is not a deletion, and the console showed a machine that had left.

Rule 2 (Membership operations are not field writes). *Join and leave change the membership relation directly. Neither is expressible as an update to a descriptive attribute, because a descriptive attribute has a default and a relation does not.*

2.3 Serving the registry is not membership

The account registry — the roster of machines, and the endpoint machines call to join or leave — runs somewhere. On a single-host console it runs on the only host, and the question does not arise. With several hosts it arises immediately and has a non-obvious answer.

Rule 3 (Registry and fleet are separable). *The host serving the registry must be removable from the fleet without being removed from service. Serving the roster and being described by it are different relations, and a design that ties them cannot express the ordinary case of a server that hosts an account’s fleet without participating in it.*

The concrete consequence in deployment: the production host serves the registry for the account while holding no machine agent and appearing nowhere in the machine list. Every other machine’s membership is mediated by a host that is not itself a member. Had the two been conflated, removing the production host from the fleet would have destroyed the roster that records the fleet — a system that cannot describe its own maintenance.

2.4 Identity gains a host index

The companion’s identity coordinate s_I answers *which agent, installation, model, owner, version am I?* Multi-host adds one term, and it is not cosmetic: *on which machine, in which account, under which label.*

s_I under multi-host

Instrument: signed registry probe, as before, plus the host’s own identity assertion and the account’s membership record for it.

Admits: machine identifier (host-asserted, L2), label (owner declaration, L5), account membership (registry record, L1).

Unmeasured reading: a machine that is running but not linked has *no* account scope, and every host-indexed coordinate below is therefore unmeasured rather than zero. An unlinked machine is not a badly-performing member; it is not a member.

Failure mode: a host whose label was never set reports the default, and two defaults are indistinguishable in a list — an identity failure the owner experiences as “which of these is the laptop?”

Deployment status

Built. Link and unlink are account operations taken from the caller’s key; a machine can be listed, labelled, and removed by label or identifier; the label has an environment override for hosts provisioned by script. The registry is served by a host outside the fleet. The label-versus-identifier failure above was found and fixed in deployment, not in design.

3 Evidence Is Host-Indexed

3.1 A measurement is about the machine that took it

The companion establishes that a coordinate is a quality measure over a store rather than the store itself, and that every coordinate publishes its instrument’s statistics. Multi-host adds a scope to the instrument. Context headroom, transcript growth, permission-store readability, tool availability, disk, and process state are all read *on a host*, by an instrument that has no access to any other host.

Design principle 1 (Host-indexed evidence). *A coordinate measured on machine h is evidence about machine h . It is not evidence about the account, and it is not evidence about machine h' . Any account-level quantity derived from it is a model, and the model must be stated.*

This is the point at which a design either does the work or quietly does the wrong thing, because the wrong thing is one line of code. An account-level dashboard needs a number. The number that presents itself is the mean over machines.

3.2 Why the fleet mean is not a health measure

Proposition 1 (The fleet-mean fallacy). *Let $s_B(h) \in [0, 1]$ be the resource-and-integrity coordinate on machine h , and let the account-level summary be $\bar{s}_B = \frac{1}{n} \sum_h s_B(h)$. Then $\bar{s}_B$ is not monotone in any operationally meaningful notion of fleet health: it can be increased without bound by adding machines that do no work, while the machine on the critical path is unchanged or degrading.*

Demonstration. Take two machines. Machine A is idle: no session, full context headroom, $s_B(A) = 1.0$. Machine B carries every live task and is at five percent headroom, $s_B(B) = 0.05$. The mean is 0.525 — a number that reads as a fleet in adequate health, and which no threshold at any plausible setting would flag.

Now link a third idle machine. Nothing about B has changed; the work it carries, its headroom, and its trend are identical. The mean rises to 0.68. The account’s reported health improved because an unrelated host was added, and the improvement is available on demand: any account can raise its fleet health arbitrarily close to 1 by linking machines that do nothing.

The failure is not a bad choice of aggregate. It is that s_B is a property of a host, health is a property of the work, and the mapping from hosts to work is exactly the information the mean discards. $\square$

Rule 4 (Account state is a vector over machines). *The account-level self-state holds one entry per member machine, each carrying its own value, sample count, recency, and unmeasured flag. No scalar summary over machines is computed for use in a decision. Where a single indicator is needed for display, it is the worst measured machine that currently carries work, and it is labelled with the machine it came from.*

The reader will recognise the argument. The companion’s Version 2.0 lineage rejects a single “self-awareness level” because a scalar re-collapses a space the vector opened and the weights are choosable [5]. Proposition 1 is that argument recurring one level up, and it is worth noting that the recursion was not anticipated: the companion applies “vector, not level” *within* an agent and then, in describing a fleet, reaches for exactly the kind of aggregate it forbids internally.

The tempting exception, refused

It is natural to answer Proposition 1 by weighting the mean by load. This is better and still wrong: it repairs the aggregate for the failure mode we happened to construct, while leaving an aggregate whose value can be moved by a quantity — assigned load — that the console itself controls. A health indicator that improves when the scheduler moves work is measuring the scheduler.

3.3 Capability is bound to a tool environment

The most consequential host-indexed coordinate is not resource integrity but capability. The companion’s s_C is a Beta–Bernoulli posterior over verified outcomes. Verified outcomes are obtained by running things, and running things depends on what is installed.

Proposition 2 (Non-portability of the capability posterior). *Let $s_C(h)$ be the capability posterior earned on host h from verified outcomes $\{o_1, \dots, o_n\}$, each obtained under h ’s tool environment E_h . Crediting $s_C(h)$ to host h' asserts a claim about outcomes under $E_{h'}$ supported by no observation under $E_{h'}$. The transfer is therefore not a conservative approximation — it is unsupported in exactly the direction that matters, since the outcomes were successes.*

Two deployment episodes make the abstraction concrete, and both were paid for.

1. **An interpreter is part of capability.** A judging step in the self-improvement loop invoked the system interpreter by name rather than the interpreter the console itself runs under. On the host where it was written, the two were the same and the step passed. On any host without that package installed globally, every improvement, promotion and rollback failed. The competence the loop had demonstrated was competence *under one environment*, and nothing in the record said so.
2. **A GPU workflow is not a workflow.** A cross-machine compute path dispatched work from a CPU host to a GPU host. The receiving host had two interpreters; only one had the numerical stack. A job that ran under one and failed under the other was, on the evidence, the same job succeeding and failing at random.

Rule 5 (What migrates when an agent moves host). *On migration of an agent from h to h' : episodic memory migrates, carrying its host provenance; procedural memory migrates marked unverified on this host until re-verified; protected memory (identity, authority, invariants) migrates unchanged, because it is account-scoped and never host-scoped; audit memory migrates and remains excluded from ordinary retrieval; the capability posterior does not migrate — the outcomes remain in the record with their host index, and the posterior on h' starts unmeasured.*

Table 2: Migration policy by store, with the reason in each case.

Store	Migrates?	Why
Episodic	yes, with provenance	Events happened; where they happened is part of the event, not a reason to discard it
Procedural	yes, marked unverified	A workflow is a hypothesis about an environment. It is worth carrying and not worth trusting
Protected	yes, unchanged	Identity, ceilings and invariants are properties of the account, which did not move
Audit	yes, still un retrievable	Accountability follows the agent; excluding it from retrieval follows it too
Capability posterior	no	Proposition 2. The outcomes are retained and host-indexed; the summary over them is re-earned
Grants scoped to a path	no	A grant naming a directory names a directory <i>on a host</i> ; the same string on another host is a different resource

The last row deserves emphasis because it is the one a hurried implementation gets wrong silently. A grant is a pair of an action and a scope, and scopes are frequently filesystem paths. Migrating a grant on a path to another host does not migrate the authority the owner intended; it manufactures authority over a different resource that happens to have the same name.

3.4 Cold start is per machine

Every new machine is a cold start, and the account will be tempted to warm it from a sibling. Rule 5 forbids it for the posterior and Proposition 2 says why. The design consequence is that the companion’s cold-start discipline — unmeasured is not zero, and “no verified outcomes yet” beats an invented prior — is not an early-life condition of the console but a *recurring* one, entered every time the owner adds a host.

Deployment status

Partly built. Coordinates are computed per installation, which gives host indexing for free on the machines that compute them, and unmeasured is honoured throughout. There is no account-level vector: the console has no aggregate at all, which is the safe half of Rule 4 and not the useful half. Migration is manual — work trees and transcripts were copied by hand — and Rule 5 describes what that migration should have been, not a procedure the console performs.

4 Channels That Can Drop

4.1 The four channels, re-typed

The companion’s four channels are unchanged in kind and changed in type. Each acquires a delivery status, and the receiver acquires the obligation to distinguish *nothing was sent* from *nothing arrived*.

Table 3: The four channels across a network. Direction and permissions are the companion’s; the last column is new.

Channel	Dir.	Carries	Delivery semantics
Typed report	up	State, evidence, uncertainty, residuals, requested action	At-least-once. Duplicates must be idempotent at the receiver; loss must be visible as staleness, never as absence
Typed directive	down	Goal, scope, criteria, budget, deadline, revocation	At-least-once with an identifier; the receiver de-duplicates. A directive that may arrive twice must not act twice
Prior bias	down	Precision, attention, evidence order, verification threshold	Best-effort. Loss is tolerable by construction: a bias may not move an estimate, so an undelivered bias changes no conclusion
Owner pre-emption	owner	Pause, resume, stop, governor	At-least-once, and acknowledged . An unacknowledged pre-emption must be reported as undelivered (§4.4)

The prior-bias row is the pleasant case and is worth pausing on, because it is a place where the companion’s governance rule pays an unexpected dividend. Prior bias may change how hard an agent looks but not what it concludes; the constraint exists for governance reasons. Its distributed consequence is that the channel needs no delivery guarantee at all — an agent that never receives a bias reaches the same conclusion more slowly. A rule adopted to keep influence honest turns out to make one of the four channels partition-tolerant for free.

4.2 Containment is satisfied by a dead link

The companion offers a leak test with an attractive property: it is checkable without understanding either agent. Project the manager’s workspace into the machine agent’s, project again into `sarsi-claude`’s, and assert containment,

$$\text{Proj}_{\text{cld}}(\text{Proj}_{\text{mac}}(W^M)) \subseteq \text{Proj}_{\text{mac}}(W^M) \subseteq W^M,$$

with a leak appearing as an item present downstream and absent upstream.

Proposition 3 (Containment is vacuous under partition). *If the link to a machine is down, that machine’s projected workspace is empty, and the empty set is contained in every set. The monotonicity check therefore passes — with its strongest possible margin — precisely when the console has no information about the agent it is checking.*

Proof. $\emptyset \subseteq X$ for all X . A partitioned machine contributes no items, so no item can appear downstream without appearing upstream. $\square$

The proposition is trivial and its consequence is not. The companion’s test is a *safety* property — nothing bad appears — and safety properties are satisfied by doing nothing. On one host, “doing nothing” was not a reachable state of the channel, so the test was adequate. Across a network it is the most common failure state, and the test is not merely weakened but actively misleading: a fleet in which every link is down reports perfect structural integrity.

Rule 6 (Every containment check carries a liveness companion). *A structural check over a remote agent's workspace is reported together with the freshness of the projection it was computed from. A check over a projection older than the reporting interval is reported as not checked, never as passed.*

4.3 Attributable silence

Rule 6 generalises, and the general form is the principle this section exists for.

Design principle 2 (Silence must be attributable). *For every expected item that is absent, the console must be able to say which of two things happened: the sender had nothing to send, or the item did not arrive. A design that cannot distinguish them will report the second as the first, because the first is the quiet case and the quiet case is the one that looks healthy.*

The single-host console already contains this principle in another form. Its error scanner matches literal failure signatures in a session's output and is documented as reporting, never ruling: a scan that matches nothing means *no signature matched*, not *the work is good*, and the renderer returns an empty string rather than a reassurance so that nothing downstream can mistake quiet for correct. Multi-host makes the same distinction structural rather than stylistic — silence now has two causes rather than one, and only one of them is benign.

Mechanically this costs a heartbeat, and the heartbeat must be separate from the reports it vouches for. A design in which the report *is* the liveness signal cannot distinguish an agent with nothing to say from an agent that cannot say it, which is the distinction being bought.

4.4 A control you cannot deliver must fail visibly

The owner's controls are the sharpest case, because they are the ones a person acts on immediately.

Rule 7 (No control reports success it did not deliver). *Pause, resume, stop, and governor changes are acknowledged by the machine that applied them. Until the acknowledgement arrives, the console shows the control as pending; if it does not arrive within the horizon, it shows undelivered and names the machine. A control is never rendered as applied on the strength of having been sent.*

The failure this prevents is specific and severe. An owner who presses stop on a task running on an unreachable machine, and is shown a stopped task, has been told the work halted. The work did not halt. Every subsequent decision — including the decision to walk away — is taken against a false statement about the world, and the false statement was generated by the console rather than by any agent. The single-host design has an analogous lesson already: pressing the interrupt key alone was insufficient because the operator resumed typing shortly afterwards, so the stop had to also stand the operator down. The correction there was that a control must change the thing it appears to change; the correction here is that a control must not *appear* to change anything until it has.

Deployment status

Not built. The deployed console is single-host for control purposes: every control acts on a session on the same machine as the console applying it, so delivery is a function call and the distinction in Rule 7 has no representation. Cross-machine reporting exists in one direction only, through an HTTP relay used for compute dispatch, and it has no acknowledgement semantics. This section is design, not description.

5 Isolation You Do Not Configure

A multi-host console invites a category of bug in which one account’s agent acts on another account’s sessions, or a staging installation drives a production fleet. The design’s position is that the strongest available isolation is the kind nobody has to switch on.

Design principle 3 (Prefer isolation the substrate already enforces). *Where the operating system, the account model, or the process model already separates two things, the console should rely on that separation rather than re-implement it as policy. Configured isolation is a statement that can be misconfigured; structural isolation is a property that cannot.*

Two deployment observations support the principle, and they fall on opposite sides of it.

Structural, and therefore free. Two users on one host each run their own terminal-multiplexer server, and the servers are per user by construction. An autopilot running as one user cannot enumerate the other user’s sessions — not because the console filters them, but because they are not visible to it. The isolation required no configuration and cannot be disabled by getting a setting wrong. It was verified by probe rather than assumed.

Configured, and therefore fragile. Two installations of the console on one host — production and staging — share that host’s session fleet. Nothing structural prevents the staging installation from attaching an operator to a production session; what prevents it is an environment flag saying this instance does not drive the machine. The flag works, and it is a policy: a host provisioned without it becomes a second driver of sessions a first driver is already steering, with two independent state directories and no way to reconcile them.

Rule 8 (Opting out of driving is one flag, not several). *The predicate “may this installation act on this machine’s sessions?” is answered in one place and consulted by every path that acts — the periodic sweep, the startup resumption of persisted duties, and any manual entry point. A flag consulted by some of them is worse than no flag, because it produces an instance that has opted out and acts anyway.*

This rule is written from a defect rather than from theory. The opt-out flag was originally read only by the periodic sweep. Startup still resumed every persisted operator and every parity watch, so an installation that had declared it did not drive the machine went back to typing into sessions the other installation was driving — silently, and only on restart. Same question, so the same predicate must answer it.

Deployment status

Built, and repaired. The per-user separation is structural and verified. The single-predicate rule is implemented: the opt-out is now consulted by the sweep and by startup resumption alike.

PART II — THE ROSTER, RE-PARTITIONED

6 Scope, Not Role, Is the Partition

The companion partitions the roster by role: manager, machine, *sarsi-claude*, learning, working-process. Multi-host adds an orthogonal question — *how many of these exist, and what is each one’s scope?* — and the answers are not uniform.

Table 4: The roster re-partitioned by scope. Cardinality is per account.

Agent	Scope	Count	Why that scope
Manager	account	1	Its dominant coordinate is organizational: owners, grants, verifiers, delegation. All of these are account properties
Learning	account	1	Its object is a <i>person’s</i> capability. A person does not become a different learner on a different host
Machine	host	n	Its object is a host: processes, resources, recovery, installed tools. Nothing it knows is true of another host
<i>sarsi-claude</i>	task	many	Its object is one session’s work. Nested inside the machine agent of the host the session runs on
Working-process	account, <i>spanning</i>	1	Its object is a workflow, and workflows cross hosts. The sole designed exception (§8)

Two entries in that table are conclusions rather than restatements, and both are worth the argument.

6.1 The learning agent must be per account

The learning agent is the companion’s special case: it models a capability that is not its own, and the design requires the two models never merge. Multi-host settles a question the companion did not have to ask, which is where that model lives.

Its object is the owner — what the owner knows, what the owner is learning, where the owner’s understanding is thin. That object is not a property of a host. An owner who works on a laptop in the morning and a workstation in the evening is one learner with one capability graph, and an implementation that instantiated a learning agent per machine would produce two partial models of one person and no principled way to combine them. Worse, it would produce them *asymmetrically*: the model on the host where the owner does difficult work would show a struggling learner, and the model on the host where the owner does routine work would show a fluent one.

Rule 9 (Person-scoped agents are account-scoped). *An agent whose object is the owner is instantiated once per account, regardless of how many machines the account owns. It receives evidence from every machine, host-indexed, and holds one model.*

6.2 The machine agent is plural, and each is a peer of the others

The companion’s machine agent is a specialist upward and a manager downward — the observation that gives “the manager role is relative”. Multi-host adds that machine agents are *peers of each other*, and the

companion’s own prohibition then applies with full force: there are no peer edges. Machine agent *A* does not message machine agent *B*. It reports upward; the manager composes a directive downward.

This is the point at which the no-peer-edges rule stops being cheap. On one host the rule cost nothing because there was one machine agent and no peer to talk to. With n machine agents and work that must move between hosts, the rule imposes a real detour, and the companion’s stated cost — the manager becomes a throughput bottleneck for cross-specialist coordination — becomes the ordinary case rather than a corner one.

The cost, stated plainly

Every cross-machine action costs two hops and one manager decision. A design that permitted machine agents to coordinate directly would be faster and would forfeit the property that makes the fleet verifiable agent by agent. This paper keeps the rule and pays the latency, with one designed exception in §8 that is constructed specifically *not* to be a peer edge.

7 The Machine SARSI Agent, Respecified

Only the deltas from the companion’s specification are given; everything unstated is unchanged.

7.1 Self-state

Table 5: Machine agent self-state under multi-host. Only changed rows are shown.

	Instantiation	Change from the companion
s_I	This host, this label, this account membership	Gains the host index and the membership relation; <i>unmeasured when unlinked</i>
s_C	What this agent can reliably do <i>on this host</i>	Posterior is host-scoped and does not migrate (Prop. 2)
s_B	This host’s context, disk, processes, store readability	Unchanged in instrument; may not be aggregated across hosts (Rule 4)
s_O	Grants applying to this host; which are path-scoped	Path-scoped grants are marked host-bound and are not portable
s_M	This agent’s history on this host, plus migrated episodes	Migrated episodes carry a foreign host index and are retrievable, but do not support host-specific claims

7.2 What it reports, and how often

Reporting contract

Heartbeat: liveness only, on a fixed interval, independent of whether there is anything to report. Separate from reports by construction (§4.3).

Typed report: state, evidence, uncertainty, residuals, requested action — at-least-once, idempotent at the manager.

Never reports: another machine’s state, or an inference about another machine. It has no instrument for either.

On rejoining after partition: reports its *current* state and the fact of the gap. It does not replay the backlog, because a queue of stale states delivered at once is indistinguishable at the manager from rapid change.

The last clause is a design decision with a cost, and the cost should be admitted: events during a partition are lost as events and survive only in the machine’s own episodic store, where an investigator can find them. The alternative — replay everything on reconnect — gives the manager a burst of history it cannot order against its own timeline, and the manager’s workspace is bounded, so the burst evicts current items in favour of stale ones.

7.3 Failure modes multi-host adds

1. **Confident about a host it is not on.** Detectable: any claim whose evidence carries a foreign host index.
2. **Reporting into an account it has left.** Prevented structurally by Rule 1: the credential determines the account, so an unlinked machine’s reports have no destination rather than the wrong one.
3. **Two agents, one host.** Two installations both driving the same host’s sessions. Prevented by Rule 8, and the failure is the one that motivated it.
4. **Migrated competence.** An agent that moved host and carried its posterior would claim verified ability it has not demonstrated in its current environment. Prevented by Rule 5; this is the failure mode the interpreter defect of §3 produced in miniature.

Deployment status

Partly built. One machine agent per host exists and is registered to the account by label. Host-scoped coordinates are computed. There is no heartbeat, no acknowledgement, no partition handling, and no migration procedure — an agent moves host by an operator copying files.

8 The Working-Process Agent: The Designed Exception

Every other agent’s object sits on one host or on none. The working-process agent’s object is a *workflow*, and a workflow in this system genuinely crosses machines: work is authored on a host with no accelerator, dispatched to a host that has one, executed there, and its result returned. That path exists in deployment and carries real jobs.

This is the one place where the no-peer-edges rule would bite hard enough to be worth examining rather than asserting, so the design examines it.

8.1 Why this is not a peer edge

The construction

The dispatching host does not address the executing host’s *agent*. It submits a job to a *queue* the executing machine agent polls, and the queue is an account-level resource whose admission is governed by the manager. The influence graph is therefore unchanged: dispatcher → manager-governed resource → executor. No agent receives a directive from a peer; each receives work from a resource its own admission rule filters.

The distinction is not a technicality. A peer edge is dangerous because it lets one agent’s state changes propagate into another’s decisions without passing an admission rule, reinstating a feedback loop that the stability argument requires to be absent. A shared queue does not have that property: the executing agent applies its own admission rule to a job exactly as it would to any other work item, and it may refuse. What crosses is a job, not an assertion — which is the same distinction the companion draws between a contract and a transcript, applied to execution rather than to conversation.

8.2 What the process agent must model that no one else does

Cross-host workflow state

Holds: which stage of a workflow is on which host; which host each artefact was produced on; which stages have host-specific prerequisites.

Does not hold: any host's resource state. It asks the manager, which asks the machine agent. A process agent that read hosts directly would be a second machine agent with no instrument.

Characteristic failure: treating a stage that succeeded on host *A* as a stage that succeeds. This is Proposition 2 in workflow form, and the mitigation is the same: a verified stage is verified *on a host*.

Deployment status

Partly built, and least conformant of the five. A cross-machine dispatch path runs real jobs over an HTTP relay. It has no queue admission governed by the manager, no per-stage host provenance, and its failures surface as raw solver output that an operator reads by hand. §8 describes where it should go, and the gap is the largest in this paper.

PART III — SURFACES AND CONSEQUENCES

9 The Chat Box, Unchanged and Harder

The surface does not change and its job gets harder, in a way that turns out to support the companion’s central observation rather than to strain it.

9.1 “Which machine?” is an organizational question

A user asks the console to run something. On one host the question of where does not arise. On several it arises every time, and the manager must answer it without the one thing that would make it easy: the manager cannot read any host’s state, because reading a host is the machine agent’s instrument and the machine agent is across a network.

The companion asserts that the manager’s dominant coordinate is organizational awareness rather than capability, and argues it from parsimony — knowing *who is authoritative about what* does not require being able to do the work. Multi-host converts that assertion into a consequence.

The observation, obtained rather than assumed

A manager separated from every host by a network holds no host’s capability or resource state. What it does hold is the membership list, the labels, the grants, and the delegation graph — all account-scoped, all local to it. Its routing must therefore be a function of ownership and authority, and where those are insufficient it must *ask* rather than infer. The companion chose organizational dominance; multi-host leaves no alternative.

9.2 Routing without host evidence

Manager routing under multi-host

Routes on: membership, labels, declared host roles, grants naming host-bound scopes, the owner’s stated preference, and the freshness of each machine’s last report.

Never routes on: an inferred belief about a host’s current capability. Stale reports are treated as unmeasured, not as current.

When it cannot decide: it asks, naming the machines by label. The cheapest honest action is a one-line question, exactly as at any other under-determined decision.

Renders: the machine a result came from, always. A result whose host is not shown is a result the user cannot reproduce.

The last line is a small requirement with a large effect on trust. In a single-host console, “it worked” is complete. In a fleet, “it worked” without a host is an incomplete claim that the user will complete incorrectly by assuming their own machine.

10 Owner Controls Across a Fleet

The single-host console gives the owner controls over a task: stop the work, turn the task off, resume it, and switch that task’s governor on or off. Multi-host adds a level above each of them, and one new obligation.

Table 6: Owner controls at two scopes. The machine column is new.

Control	Per task (built)	Per machine (designed)
Stop	Interrupt the turn, stand the operator down, lapse on a horizon	Stand down every operator on that host; same lapse, so a forgotten machine-stop cannot strand a fleet
Off	End the task, keep the record so it is resumable	Suspend the machine’s participation without unlinking it; membership is not a power switch
Resume	Restart with the same name, workspace and ceiling	Resume the host’s duties; each session resumes individually and independently
Governor	Add or remove the task’s own enforcement hook	<i>No machine-level equivalent, deliberately (§10.1)</i>
Leave	—	Unlink: the machine stops being a member. Distinct from off, and not reversible by resume

10.1 Why there is no machine-level governor

The per-task governor is an enforcement hook inside the task’s own workspace, and its presence is its state — there is no separate flag that could disagree with it. A machine-level governor switch would be a different kind of object: a setting that claims to govern tasks it does not live inside, which could be on while a task’s own hook is missing, and which would therefore be capable of reporting a governed fleet containing an ungoverned task.

Rule 10 (No enforcement claim without the enforcing artefact). *The console does not offer a control whose state could differ from the enforcement it describes. Governor state is read from the artefact that enforces it, per task, and a fleet-level view is an aggregation of those readings — never a setting of its own.*

A machine-level *view* is fine and is required: the owner should be able to see that some task on a host is ungoverned. What is refused is a machine-level *switch*, because the switch would have to be stored somewhere other than where enforcement lives, and the two would eventually disagree.

10.2 The new obligation

Every control in Table 6 is subject to Rule 7: it is pending until acknowledged and undelivered if it is not. This is the single largest behavioural difference the owner will notice between the single-host console and the fleet console, and it will be experienced as the interface becoming slower and less certain. It is more honest, and the honesty is the feature: the previous interface was fast and certain because it was describing a function call.

11 Failure Modes This Version Adds

Two of these — *two drivers, one host* and *imported competence* — have already occurred in deployment and are described from the incident rather than from imagination. The others are predicted, and the honest status is that a predicted failure mode with a stated detection is a hypothesis about a system, not a property of one.

Table 7: Failure modes introduced by multi-host, with detection.

Failure	What it looks like	Detection
Health by dilution	Fleet health improves when an idle machine is linked	Rule 4: there is no fleet scalar to inflate
Integrity by partition	Every structural check passes, and every link is down	Rule 6: freshness reported with every check
Phantom control	A control shows as applied on an unreachable machine	Rule 7: pending until acknowledged
Imported competence	An agent claims verified ability it earned elsewhere	Rule 5; foreign host index on the evidence
Path-grant laundering	A grant on a path is honoured on a different host	Grants naming paths are host-bound and do not migrate
Two drivers, one host	Two installations steer the same sessions	Rule 8: one predicate, consulted everywhere
Backlog burst	A rejoining machine floods the manager with stale state	Report current state and the gap; never replay
Silent orphan	A machine left the account and keeps working	Rule 1: reports have no destination

12 Build Order

The order is chosen so that each item is enforceable when it lands, rather than correct in principle and unenforced.

1. **Heartbeat, separate from reporting.** Everything downstream needs attributable silence (§4.3), and it cannot be retrofitted onto a channel where the report is the liveness signal.
2. **Acknowledged controls.** Rule 7. Before any control acts across a host boundary, because the first unacknowledged stop is the one that costs trust.
3. **Account state as a vector.** Rule 4. Land it before any fleet-level display exists, since the display is what creates the demand for a scalar.
4. **Freshness on every structural check.** Rule 6. Cheap, and it turns Proposition 3 from a live hazard into a reported condition.
5. **Migration as a procedure.** Rule 5 with per-store policy, replacing the manual copy.
6. **Queue-mediated cross-host work.** §8, with manager-governed admission — the largest item, and the one that must not be started before 1 and 2, since it depends on both.

Item 6 is deliberately last despite being the most valuable, because it is the item whose premature implementation creates a peer edge. Built before the queue’s admission is governed, cross-host dispatch is exactly the direct agent-to-agent path the architecture forbids, and it would be very difficult to remove afterwards.

13 Conclusion

The companion design’s five agents, four channels, and one chat box survive contact with a fleet of machines. What does not survive is the arithmetic around them: a mean over hosts is not a health measure, a containment check over a dead link is not an integrity result, and a capability posterior is not a portable asset. Each of

these is a place where the single-host design had a free answer and the multi-host design has to choose one.

The re-partition by scope — two account-scoped agents, n host-scoped, many task-scoped, and exactly one that spans — is the structural contribution, and the spanning agent is the interesting one precisely because it is the exception. An architecture with no exceptions to its own coordination rule has usually not met a workload that needed one; an architecture that grants the exception without construction has usually given away the property the rule protected. The queue-mediated construction in §8 is the attempt to do neither.

The through-line with the rest of the corpus is one rule stated at yet another scale. The system-scale paper argues that authority closure needs an evidence base; the agent-scale paper supplies it and requires that the promoter differ from the proposer; the fleet-scale paper extends this to what one agent may assert about another; the console design carries it to a product surface as *memory does not cross agents*. This paper carries it one step further, to *evidence does not cross machines* — and finds, at the end, that the reason is the same reason each time. Evidence means something because of the conditions under which it was gathered, and a boundary that discards those conditions converts a measurement into a claim.

A Invariant Checklist: Multi-Host Additions

The companion’s invariants hold unchanged. These are additional, and each is stated so that it can be checked mechanically.

- M1.** The account of any membership operation is derived from the caller’s credential and never read from the request body.
- M2.** Join and leave modify the membership relation; neither is implemented as a write to a descriptive field.
- M3.** The host serving the registry can be removed from the fleet without loss of registry service.
- M4.** No decision consumes a scalar aggregate of a host-indexed coordinate across machines.
- M5.** Every structural check over a remote workspace is reported with the freshness of the projection it used; a stale projection yields *not checked*, never *passed*.
- M6.** Every absent expected item is attributable to *not sent* or *not delivered*.
- M7.** Liveness is signalled by a channel distinct from the reporting channel.
- M8.** No control renders as applied before it is acknowledged by the machine that applied it.
- M9.** A capability posterior is never credited to a host other than the one whose outcomes produced it.
- M10.** A grant whose scope names a filesystem path is host-bound and does not migrate.
- M11.** No agent sends a directive to a peer; cross-host work passes through a manager-governed resource.
- M12.** The predicate “may this installation act on this machine’s sessions?” is defined once and consulted by every acting path, including startup resumption.
- M13.** No control exists whose stored state could differ from the artefact that enforces it.
- M14.** A rejoining machine reports current state and the fact of a gap; it does not replay a backlog.

B Deployment Status Summary

References

- [1] Chengshuai Yang. One chat box, five agents: Designing the manager-fronted SARSI console — per-agent self-awareness, nested workspaces, and the four channels by which agents affect each other. Preprint. Referred to throughout as “the companion design”, 2026.

Table 8: What is built, specified, or neither, as of this writing.

Element	Status	Note
Account with a machine list	built	Link, unlink, list, label, remove
Account from credential, not body	built	Both directions
Registry served from outside the fleet	built	Production host serves; is not a member
Per-host machine agent	built	One per host, registered by label
Host-scoped coordinates	built	Computed per installation
Single opt-out predicate	built	Repaired after the startup-resumption defect
Per-user structural isolation	built	Property of the substrate; verified by probe
Account state as a vector	not built	No account-level aggregate of any kind exists
Heartbeat separate from reports	not built	—
Acknowledged controls	not built	Controls are same-host function calls today
Freshness on structural checks	not built	—
Migration procedure	not built	Manual file copy
Queue-mediated cross-host work	partly	Dispatch path runs; admission is not manager-governed
Machine-level view of governor state	not built	Per-task reading exists and is authoritative

- [2] Chengshuai Yang. Functional self-awareness for hierarchical SARSI multi-agent systems: Version 3.0 — a brain-inspired manager–specialist architecture with bidirectional influence, nested workspaces, and safe user interruptibility. Preprint. Referred to throughout as “Version 3.0”, 2026.
- [3] Chengshuai Yang. Functional self-awareness for SARSI agents: Version 2.0 (history-aware revision) — selective episodic encoding, replay, consolidation, and bounded retrieval. Preprint, 2026.
- [4] Chengshuai Yang. A manager is not a controller: Global-workspace coordination for SARSI fleets. Companion paper on coordination semantics, 2026.
- [5] Chengshuai Yang. Functional self-awareness for SARSI agents: Version 2.0 (corrected and expanded). Preprint. The single-agent architecture Version 3.0 rests on, 2026.
- [6] Chengshuai Yang. Functional self-awareness for hierarchical SARSI multi-agent systems: Version 3.1 — deployment edition: What building it changed. Implementation-grounded revision of Version 3.0, 2026.